

雷达数据存储系统

部署运维手册 V1.0

4节点MinIO分布式集群 + 双Nginx负载均衡

Keepalived VIP高可用 + 分层存储 + 自动生命周期管理

支持 20 ~ 50 部雷达平滑扩展

文档版本：1.0

更新日期：2026年6月19日

目录

1. 系统概述

1.1 架构设计

1.2 核心特性

1.3 数据流向

2. 环境要求

2.1 硬件要求

2.2 软件要求

2.3 网络规划

3. 快速部署

3.1 目录结构

3.2 部署步骤

3.3 访问入口

4. 组件详解

4.1 MinIO分布式集群

4.2 Nginx负载均衡

4.3 Keepalived VIP

4.4 分层存储

4.5 生命周期管理

4.6 雷达模拟器

4.7 监控面板

5. 运维管理

5.1 管理命令

5.2 水平扩展

5.3 日常监控

5.4 日志查看

5.5 数据备份

6. 故障排除

6.1 容器启动失败

[6.2 MinIO集群离线](#)

[6.3 VIP漂移不生效](#)

[6.4 分层存储异常](#)

[6.5 Nginx配置验证](#)

[7. 安全建议](#)

[8. 附录](#)

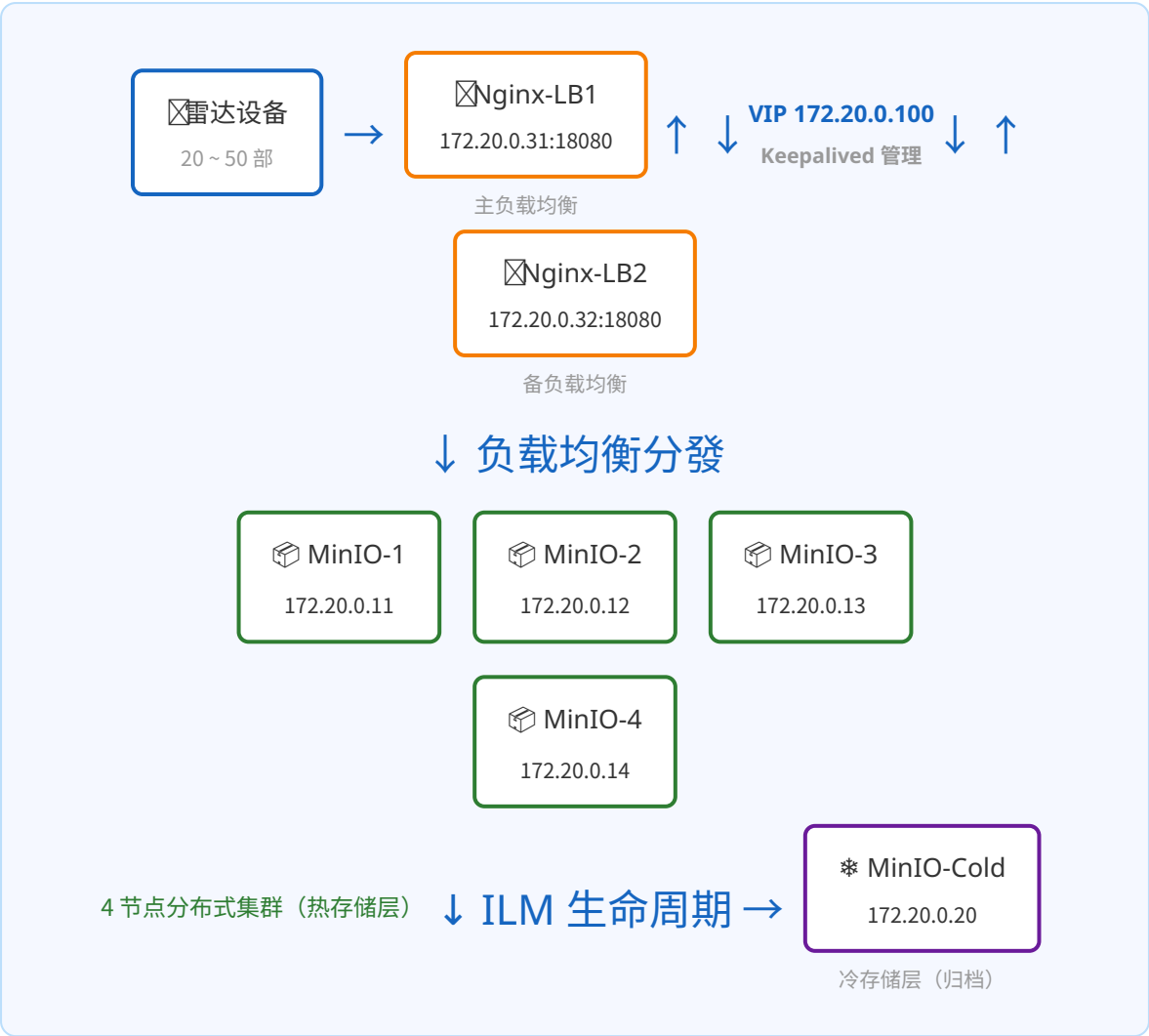
[8.1 配置文件索引](#)

[8.2 常用命令速查](#)

1. 系统概述

本系统是一套面向雷达数据存储场景的高可用分布式存储解决方案，基于 **4节点MinIO集群** 提供对象存储服务，通过 **双Nginx负载均衡 + Keepalived VIP漂移** 实现高可用接入，并配备 **分层存储** 和 **自动生命周期管理** 能力。

1.1 架构设计



1.2 核心特性

特性	说明
高可用	

	4节点MinIO分布式集群 + 双Nginx负载均衡 + Keepalived VIP漂移
自动分层	热数据保留在本地集群，冷数据自动过渡到归档存储
生命周期管理	基于时间的自动过期删除、存储类转换，无需人工干预
弹性扩展	支持 20~50 部雷达，增加存储节点即可线性扩展容量和性能
数据安全	MinIO Erasure Coding 数据保护、桶版本控制、对象锁定
实时监控	Web 监控面板，实时查看集群状态、存储使用量和数据吞吐

1.3 数据流向

1. **写入路径**：雷达模拟器 → Nginx负载均衡器（S3 API） → MinIO集群节点 → 分布式存储
2. **读取路径**：客户端 → VIP地址 → Nginx主节点（故障时自动切换备节点） → MinIO集群
3. **生命周期**：实时数据写入 rada-data 桶 → 7天后过渡到冷存储COLD-TIER-S3 → 90天后过期清理
4. **高可用切换**：Keepalived 每5秒检测MinIO集群健康，主节点故障时VIP自动漂移至备节点

2. 环境要求

2.1 硬件要求

部署规模	雷达数	CPU	内存	磁盘
测试环境	20部	4核	8 GB	50 GB
小型生产	20~30部	8核	32 GB	500 GB SSD
中型生产	30~50部	16核	64 GB	1 TB NVMe

2.2 软件要求

组件	版本要求	说明
Docker	≥ 24.x	容器运行环境
Docker Compose	≥ 2.20	容器编排工具
MinIO	RELEASE.2024-01-11T07-46-16Z	对象存储服务
Nginx	1.25-alpine	负载均衡器
Keepalived	latest (Alpine)	VIP漂移管理
Python	3.11+	模拟器和监控脚本运行环境

提示：生产环境建议提前拉取镜像避免部署时等待：

```
docker pull minio/minio:RELEASE.2024-01-11T07-46-16Z
docker pull nginx:1.25-alpine
docker pull minio/mc:latest
docker pull python:3.11-alpine
docker pull alpine:3.18
```

2.3 网络规划

组件	内部IP	外部端口	协议
MinIO Node1	172.20.0.11	9011 (S3) / 19111 (Console)	HTTP
MinIO Node2	172.20.0.12	9012 (S3) / 19112 (Console)	HTTP
MinIO Node3	172.20.0.13	9013 (S3) / 19113 (Console)	HTTP
MinIO Node4	172.20.0.14	9014 (S3) / 19114 (Console)	HTTP
MinIO Cold	172.20.0.20	9020 (S3) / 19220 (Console)	HTTP
Nginx LB1	172.20.0.31	18080 (S3) / 18081 (Console) / 19090 (Status)	HTTP
Nginx LB2	172.20.0.32	28080 (S3) / 28081 (Console) / 29090 (Status)	HTTP
Keepalived Master	172.20.0.41	-	VRRP
Keepalived Backup	172.20.0.42	-	VRRP
VIP (虚拟IP)	172.20.0.100	18080 / 18081	HTTP
监控面板	172.20.0.70	8888	HTTP

注意：内部IP由Docker桥接网络自动分配，上述为固定分配值。VIP 172.20.0.100 由Keepalived管理，默认绑定在主Nginx节点上，主节点故障时自动漂移到备节点。

3. 快速部署

3.1 目录结构

```

radar-storage/
├── docker-compose.yml      # 主编排文件（所有服务定义）
├── manage.sh               # 统一管理脚本
├── minio/
│   ├── node1~4/           # 4个分布式节点数据/配置目录
│   └── cold-tier/         # 冷存储层数据/配置目录
├── nginx/
│   ├── nginx1/            # 主负载均衡器配置
│   │   ├── nginx.conf
│   │   └── conf.d/
│   │       ├── minio-s3.conf      # S3 API代理配置
│   │       └── status.conf        # 状态监控页面
│   └── nginx2/            # 备负载均衡器配置（结构同上）
├── keepalived/
│   ├── keepalived-master.conf    # 主节点VIP配置
│   ├── keepalived-backup.conf    # 备节点VIP配置
│   ├── check_minio.sh            # 健康检查脚本
│   └── notify.sh                 # 状态切换通知脚本
├── ilm/
│   ├── ilm-rule-hot-to-warm.json  # ILM规则：热→冷（7天）
│   ├── ilm-rule-warm-to-cold.json # ILM规则：温→冷（30天）
│   ├── ilm-rule-backup.json      # ILM规则：备份过期（90天）
│   └── radar-policy.json          # MinIO访问策略
└── scripts/
    ├── init_minio.sh              # MinIO集群初始化脚本
    ├── radar_simulator.py         # 雷达数据模拟器
    ├── monitor_server.py          # 监控面板服务
    └── health_check.py            # 健康检查工具
  
```

3.2 部署步骤

3.2.1 下载项目

```
cd /workspace/radar-storage
```


3.2.2 一键启动

```
./manage.sh start
```

该命令将依次完成：

- 1. 自动创建 Docker 网络和持久化数据卷
- 2. 拉取所有镜像（第一次启动时）
- 3. 启动4个MinIO分布式节点（自动形成集群）
- 4. 启动冷存储 MinIO 实例
- 5. 启动双 Nginx 负载均衡器
- 6. 启动 Keepalived 主备节点（建立VIP）
- 7. 执行初始化脚本：创建桶、配置分层存储、设置ILM策略、创建服务账号
- 8. 启动雷达模拟器（自动写入数据）
- 9. 启动监控面板

3.2.3 验证部署

```
# 查看所有容器状态
./manage.sh status

# 运行健康检查
./manage.sh health

# 查看MinIO集群信息
docker exec radar-mc-init mc admin info hot
```

3.3 访问入口

服务	地址	凭证
MinIO Console（节点1）	http://localhost:19111	radaradmin / RadarAdmin@2024!
MinIO Console（节点2）	http://localhost:19112	同上
S3 API（主负载均衡）	http://localhost:18080	-
S3 API（备负载均衡）	http://localhost:28080	-
Nginx状态页（主）		-

	http://localhost:19090/ status	
监控面板	http://localhost:8888	-
VIP入口	http://172.20.0.100:18080	-

提示：雷达模拟器使用服务账号 `radar-simulator / RadarSim@2024!` 通过Nginx负载均衡写入数据，测试客户端连接时也应使用此地址。

4. 组件详解

4.1 MinIO分布式集群

节点配置

4个MinIO节点通过 `http://minio-node{1...4}/data` 组成分布式集群。所有节点必须具有完全一致的环境变量，否则集群启动会报错。

```
# 关键环境变量
MINIO_ROOT_USER=radaradmin
MINIO_ROOT_PASSWORD=RadarAdmin@2024!
MINIO_SITE_REGION=cn-east-1
MINIO_PROMETHEUS_AUTH_TYPE=public
```

启动命令

```
server --console-address ":9001" http://minio-node{1...4}/data
```

使用 `{1...4}` 通配符语法自动识别集群成员。

健康检查

Docker healthcheck 使用 bash 内置TCP连接测试：

```
bash -c "exec 3<>/dev/tcp/localhost/9000"
```

存储桶规划

桶名	用途	标签	配额
radar-data	实时雷达数据	radar-type=real-time, retention=hot	100 GB
radar-archive	归档数据	radar-type=archive, retention=warm	200 GB
radar-backup	备份数据	radar-type=backup, retention=warm	无限制

4.2 Nginx负载均衡

配置项	值	说明
负载均衡算法	least_conn	最少连接数，适合大文件传输场景
连接保持	keepalive 32	复用后端连接，减少TCP开销
超时设置	connect 30s / send 300s / read 300s	适应雷达大文件传输
失败重试	max_fails=3, fail_timeout=30s	节点故障自动摘除
缓冲区	proxy_buffering off	禁用缓冲，适配流式传输
最大请求体	client_max_body_size 0	无限制，支持超大文件

主备Nginx差异：两个节点的Nginx配置基本相同，区别在于响应头 `X-Node` 分别标识 `nginx-lb1` 和 `nginx-lb2`，便于排查请求走了哪个节点。

4.3 Keepalived VIP漂移

配置项	主节点	备节点
优先级	150	100
VRRP ID	51	51
通告间隔	1秒	1秒
抢占模式	开启（延迟10秒）	关闭（nopreempt）
健康检查脚本	check_minio.sh（5秒间隔）	同左

切换条件：MinIO集群超过半数节点宕机 → 健康检查失败 → 优先级降低 → VIP漂移到备节点。

注意：在Docker中运行Keepalived需要 `privileged: true` 和 `NET_ADMIN`, `NET_RAW`, `NET_BROADCAST` 权限。

4.4 分层存储

系统配置了 **COLD-TIER-S3** 远程存储层，将MinIO冷存储实例作为归档目标：

```
mc admin tier add minio hot COLD-TIER-S3 \
  --endpoint http://minio-cold:9000 \
  --access-key radaradmin \
  --secret-key 'RadarAdmin@2024!' \
  --bucket radar-archive-cold \
  --region cn-east-1
```

存储分层结构



4.5 生命周期管理

系统配置了3条ILM规则，实现数据全生命周期自动管理：

桶	规则	触发条件	动作
radar-data	hot-to-warm-7d	非当前版本 7天	过渡到 COLD-TIER-S3
radar-data	hot-to-warm-7d	当前版本 90天	过期删除
radar-archive	warm-to-cold-30d	非当前版本 30天	过渡到 COLD-TIER-S3
radar-archive	warm-to-cold-30d	当前版本 365天	过期删除
radar-backup	backup-expire-90d	当前版本 90天	过期删除
radar-backup	backup-expire-90d	非当前版本 30天	过期删除

4.6 雷达模拟器

模拟器支持4种雷达类型，按加权概率随机分配：

雷达类型	数据量	上报间隔	S3前缀	优先级
气象雷达 weather	1~5 MB	5分钟	weather/	高
监视雷达 surveillance	5~20 MB	1分钟	surveillance/	关键

成像雷达 imaging	10~50 MB	10分钟	imaging/	中
跟踪雷达 tracking	0.5~2 MB	10秒	tracking/	关键

数据存储路径格式：`{prefix}/{radar-id}/YYYY/MM/DD/HH/MM/{radar-id}`
`_timestamp_uuid.radar`

4.7 监控面板

Flask Web监控服务，提供：

- 集群概览：桶数量、对象总数、总数据量
- 存储桶详情：每个桶的对象数、数据量、标签
- 最近上传活动日志
- JSON API端点供外部监控系统集成

集成建议：监控面板提供 `/api/status` 和 `/api/buckets` API，可对接 Prometheus + Grafana 或 Zabbix 等企业监控平台。

5. 运维管理

5.1 管理命令

```
# 系统管理
./manage.sh start      # 启动系统
./manage.sh stop       # 停止系统
./manage.sh restart    # 重启系统
./manage.sh status     # 查看容器状态
./manage.sh health     # 健康检查

# 扩展运维
./manage.sh scale 35    # 调整雷达数量 (1-50)
./manage.sh logs minio-node1 # 查看指定服务日志
./manage.sh clean      # 清理所有数据卷 (慎用! )
```

5.2 水平扩展

扩展MinIO节点

从4节点扩展到更多节点（如6节点），只需在 docker-compose.yml 中新增节点定义，启动命令改为：

```
server --console-address ":9001" http://minio-node{1...N}/data
```

扩展雷达数量

```
# 调整雷达模拟器数量
./manage.sh scale 50    # 立即扩展到50部雷达
```

5.3 日常监控

```
# 实时资源监控
docker stats $(docker compose ps -q)

# MinIO集群状态
docker exec radar-mc-init mc admin info hot
```

```
docker exec radar-mc-init mc admin disk hot

# MinIO集群性能
docker exec radar-mc-init mc admin perf hot

# 查看存储桶使用量
docker exec radar-mc-init mc du hot --recursive

# 查看ILM策略
docker exec radar-mc-init mc ilm ls hot/radar-data
```

5.4 日志查看

```
# 查看所有日志（实时）
docker compose logs -f

# 查看特定服务日志
docker compose logs -f minio-node1
docker compose logs -f radar-simulator
docker compose logs -f nginx-lb1

# 仅查看最近50行
docker compose logs --tail=50 minio-node1
```

5.5 数据备份

```
# MinIO桶备份到冷存储（手动触发）
docker exec radar-mc-init mc mirror hot/radar-data cold/radar-archive-cold

# 使用mc创建桶快照
docker exec radar-mc-init mc cp --recursive hot/radar-data cold/radar-backup/$(date +%Y%m%d)

# 导出MinIO配置
docker exec radar-mc-init mc admin config get hot
```


6. 故障排除

6.1 容器启动失败

现象	原因	解决方案
MinIO节点 unhealthy	镜像版本不对或环境变量不一致	确保所有节点有相同的 <code>MINIO_ROOT_USER</code> 等环境变量
keepalived 反复重启	脚本挂载为只读，chmod失败	挂载脚本卷时去掉 <code>:ro</code> 标记
端口冲突	外部端口已被占用	修改 <code>docker-compose.yml</code> 中的端口映射

修复步骤

```
# 1. 查看容器日志定位问题
docker logs radar-minio-node1

# 2. 清理并重启
./manage.sh clean && ./manage.sh start

# 3. 或单独重建特定容器
docker compose up -d --force-recreate minio-node1
```

6.2 MinIO集群离线

检查命令	预期输出	异常处理
<code>mc admin info hot</code>	4 Online	检查网络连通性: <code>ping minio-node2</code>
<code>docker ps grep minio</code>	4个节点 healthy	重启故障节点: <code>docker restart radar-minio-nodeX</code>
<code>curl localhost:9011/minio/health/live</code>	HTTP 200	检查磁盘空间: <code>df -h</code>

6.3 VIP漂移不生效

```
# 检查Keepalived进程
docker exec radar-keepalived-master ps aux | grep keepalived

# 检查VIP是否绑定
docker exec radar-keepalived-master ip addr show eth0

# 查看Keepalived日志
docker logs radar-keepalived-master

# 手动测试健康检查脚本
docker exec radar-keepalived-master sh /usr/local/bin/check_minio.sh
```

6.4 分层存储异常

```
# 查看已配置的存储层
docker exec radar-mc-init mc admin tier ls hot

# 检查分区状态
docker exec radar-mc-init mc ilm ls hot/radar-data

# 手动触发ILM扫描（MinIO每小时自动扫描）
docker exec radar-mc-init mc ilm ls hot/radar-data --verbose
```

6.5 Nginx配置验证

```
# 检查Nginx配置语法
docker exec radar-nginx-lb1 nginx -t

# 查看Nginx访问日志
docker exec radar-nginx-lb1 cat /var/log/nginx/minio-s3-access.log

# 查看Nginx错误日志
docker exec radar-nginx-lb1 cat /var/log/nginx/error.log
```

7. 安全建议

#	建议	说明
1	修改默认密码	部署后立即修改 <code>radaradmin</code> 的密码，生产环境使用强密码策略
2	启用TLS	为MinIO和Nginx配置SSL证书，使用HTTPS协议传输雷达数据
3	网络隔离	生产环境将MinIO集群部署在私有网络，仅暴露Nginx负载均衡端口
4	访问控制	为不同雷达设备创建独立服务账号，遵循最小权限原则
5	审计日志	启用MinIO审计日志 (<code>MINIO_AUDIT_WEBHOOK</code>)，记录所有数据访问行为
6	版本管理	对关键桶启用版本控制 (<code>mc version enable</code>)，防止数据误删
7	定期巡检	每周执行 <code>./manage.sh health</code> 检查系统健康，每月检查磁盘容量趋势
8	数据加密	启用MinIO服务端加密 (<code>MINIO_KMS</code>)，确保存储数据加密

8. 附录

8.1 配置文件索引

文件	用途	关键配置项
docker-compose.yml	服务编排定义	12个服务定义、网络、数据卷
nginx/nginx1/nginx.conf	Nginx主配置	worker_connections, upstream定义
nginx/nginx1/conf.d/minio-s3.conf	S3 API代理规则	端口18080, least_conn, 超时设置
keepalived/keepalived-master.conf	VIP主节点	priority 150, VRRP ID 51
keepalived/check_minio.sh	健康检查	检测4个节点, 满足多数在线即正常
ilm/ilm-rule-hot-to-warm.json	热→冷过渡(7天)	StorageClass: COLD-TIER-S3
scripts/radar_simulator.py	雷达数据模拟	4种雷达类型, 自动分片上传
scripts/monitor_server.py	监控Web面板	Flask, JSON API, 自动刷新

8.2 常用命令速查

```
#
# | 管理类命令 |
#
./manage.sh start      # 启动全部服务
./manage.sh stop       # 停止全部服务
./manage.sh restart    # 重启全部服务
./manage.sh status     # 查看容器状态
./manage.sh health     # 完整健康检查
./manage.sh scale N    # 调整雷达数量
./manage.sh logs [service] # 查看日志

#
# | MinIO 命令类 |
#
docker exec radar-mc-init mc admin info hot # 集群信息
```

```
docker exec radar-mc-init mc admin disk hot      # 磁盘状态
docker exec radar-mc-init mc admin perf hot      # 性能测试
docker exec radar-mc-init mc ls hot/radar-data   # 列出对象
docker exec radar-mc-init mc du hot --recursive # 存储用量
docker exec radar-mc-init mc ilm ls hot/radar-data # ILM策略
```

```
#
# | 监控诊断类 |
#
```

```
docker stats $(docker compose ps -q)           # 实时资源监控
curl http://localhost:18080/minio/health/live   # S3健康检查
curl http://localhost:8888/api/status           # API状态查询
curl http://localhost:19090/status              # Nginx状态
```

```
#
# | 故障恢复类 |
#
```

```
docker compose logs -f minio-node1             # 节点日志
docker compose up -d --force-recreate [svc]     # 强制重建
docker compose down -v && docker compose up -d  # 完全重置
```

快速参考：所有MinIO命令使用 `docker exec radar-mc-init mc <command>` 格式执行，mc-init容器保持运行，可作为MinIO管理代理使用。